Assignment 2

JavaScript Fundamentals

Technology: JavaScript (ES6+)
Purpose: Understand programming fundamentals and make webpages interactive.

---

1. Introduction to JavaScript

JavaScript is a programming language used to add functionality and interactivity to websites.

HTML vs CSS vs JavaScript

Technology	Purpose
HTML	Structure
CSS	Design and layout
JavaScript	Behaviour and interactivity

Example:

```
None
<h1 id="heading">Welcome</h1>
<button onclick="changeText()">Click Me</button>

<script>
    function changeText() {
        document.getElementById("heading").textContent = "Hello
Divya!";
    }
</script>
```

Ways to include JavaScript

1. Inline JavaScript

```
None
<button onclick="alert('Hello!')">Click</button>
```

2. Internal JavaScript

```
None
<script>
    console.log("Hello JavaScript");
</script>
```

3. External JavaScript

HTML:

```
None
<script src="script.js" defer></script>
```

script.js:

```
None
console.log("External JavaScript");
```

External files are preferred for maintainability and reuse.

---

2. Variables

Variables store data.

JavaScript provides three main variable declaration keywords.

```
None
var name = "Divya";
let age = 25;
const country = "India";
```

Keyword	Description
var	Function-scoped; can be redeclared and reassigned
let	Block-scoped; can be reassigned
const	Block-scoped; cannot be reassigned

Example:

```
None
let marks = 80;
marks = 90;

const pi = 3.14;

// pi = 4; // Error
```

Remember: Prefer `const` by default. Use `let` when reassignment is needed. Avoid `var` in modern JavaScript unless required.

---

3. JavaScript Data Types

Primitive types

Type	Example
String	`"Hello"`
Number	`100`, `10.5`
Boolean	`true`, `false`
Undefined	`let x;`
Null	`let x = null;`
BigInt	`123456789n`
Symbol	`Symbol("id")`

Example:

```
None
let name = "Divya";
let age = 25;
let isStudent = true;
let result;
let address = null;

console.log(typeof name);
console.log(typeof age);
```

```
console.log(typeof isStudent);
```

Note: `typeof null` returns `"object"` due to a historical JavaScript behaviour.

Non-primitive type

Objects include objects, arrays and functions.

```
None
let student = {
    name: "Divya",
    age: 25
};
```

---

4. Operators

Arithmetic operators

```
None
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
console.log(a ** b); // 1000
```

Assignment operators

```
None
let x = 10;

x += 5; // 15
x -= 2; // 13
x *= 2; // 26
x /= 2; // 13
```

Comparison operators

```
None
console.log(10 == "10");  // true
console.log(10 === "10"); // false
console.log(10 != "10"); // false
console.log(10 !== "10"); // true

console.log(10 > 5);
console.log(10 < 5);
console.log(10 >= 10);
```

Important: `==` compares after type conversion when necessary. `===` compares without type coercion.

Logical operators

```
None
let age = 25;

console.log(age >= 18 && age <= 60);
console.log(age < 18 || age > 60);
console.log(!(age < 18));
```

Operator	Meaning
`&&`	AND
`` ` ` ``	OR
`!`	NOT

---

5. Conditional Statements

Conditional statements execute code based on conditions.

if

```
None
let age = 20;

if (age >= 18) {
    console.log("Eligible to vote");
}
```

if...else

```
None
let marks = 45;

if (marks >= 50) {
    console.log("Pass");
} else {
    console.log("Fail");
}
```

else if

```
None
let marks = 85;

if (marks >= 90) {
    console.log("Grade A");
} else if (marks >= 75) {
    console.log("Grade B");
} else if (marks >= 50) {
    console.log("Grade C");
} else {
    console.log("Fail");
}
```

switch

```
None
let day = 2;

switch (day) {
    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    default:
```

```
        console.log("Invalid day");
}
```

---

6. Loops

Loops execute code repeatedly.

for loop

```
None
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

while loop

```
None
let i = 1;

while (i <= 5) {
    console.log(i);
    i++;
}
```

do...while loop

```
None
let i = 1;

do {
    console.log(i);
    i++;
} while (i <= 5);
```

for...of

Used to iterate over iterable values such as arrays.

```
None
let fruits = ["Apple", "Mango", "Orange"];
```

```
for (let fruit of fruits) {
    console.log(fruit);
}
```

for...in

Used to iterate over enumerable property keys of an object.

```
None
let student = {
    name: "Divya",
    age: 25
};

for (let key in student) {
    console.log(key, student[key]);
}
```

break and continue

```
None
for (let i = 1; i <= 5; i++) {
    if (i === 3) {
        break;
    }

    console.log(i);
}
```

`break` stops the loop.

```
None
for (let i = 1; i <= 5; i++) {
    if (i === 3) {
        continue;
    }

    console.log(i);
}
```

`continue` skips the current iteration.

7. Functions

Functions are reusable blocks of code.

Function declaration

```
None
function greet() {
    console.log("Hello!");
}

greet();
```

Parameters and arguments

```
None
function greet(name) {
    console.log("Hello " + name);
}

greet("Divya");
```

- `name` is a parameter.
- `"Divya"` is an argument.

Return statement

```
None
function add(a, b) {
    return a + b;
}

let result = add(10, 20);

console.log(result); // 30
```

Function expression

```
None
const multiply = function(a, b) {
    return a * b;
```

```
};

console.log(multiply(5, 4));
```

Arrow function

```
None
const add = (a, b) => {
    return a + b;
};

console.log(add(10, 20));
```

Short form:

```
None
const square = number => number * number;

console.log(square(5)); // 25
```

---

8. Arrays

An array stores multiple values in one variable.

```
None
let fruits = ["Apple", "Mango", "Orange"];

console.log(fruits[0]); // Apple
console.log(fruits[1]); // Mango
```

Array indexes start at zero.

Update array values

```
None
fruits[1] = "Banana";
```

Common array properties and methods

```
None
let numbers = [10, 20, 30];

numbers.push(40);    // Add at end
numbers.pop();       // Remove from end
numbers.unshift(5);  // Add at beginning
numbers.shift();     // Remove from beginning

console.log(numbers.length);
```

Other useful methods

```
None
let numbers = [1, 2, 3, 4, 5];

console.log(numbers.includes(3));
console.log(numbers.indexOf(4));
console.log(numbers.slice(1, 3));
```

map()

Creates a new array by transforming each element.

```
None
let numbers = [1, 2, 3];

let doubled = numbers.map(n => n * 2);

console.log(doubled); // [2, 4, 6]
```

filter()

Creates a new array containing elements that satisfy a condition.

```
None
let numbers = [10, 15, 20, 25];

let result = numbers.filter(n => n > 15);

console.log(result); // [20, 25]
```

reduce()

Combines array elements into one result.

```
None
let numbers = [10, 20, 30];

let total = numbers.reduce((sum, n) => sum + n, 0);

console.log(total); // 60
```

---

9. Objects

Objects store data in key-value pairs.

```
None
let student = {
    name: "Divya",
    age: 25,
    course: "JavaScript"
};
```

Access properties

```
None
console.log(student.name);
console.log(student["age"]);
```

Update properties

```
None
student.age = 26;
student.city = "Kerala";
```

Object method

```
None
let student = {
    name: "Divya",

    greet: function() {
        return "Hello " + this.name;
    }
};
```

```
console.log(student.greet());
```

`this` refers to the object in this method call.

---

10. Strings and Numbers

String methods

```
None
let name = "JavaScript";

console.log(name.length);
console.log(name.toUpperCase());
console.log(name.toLowerCase());
console.log(name.includes("Script"));
console.log(name.slice(0, 4));
console.log(name.replace("Java", "Type"));
```

Template literals

Use backticks to insert expressions into strings.

```
None
let name = "Divya";
let age = 25;

console.log(`My name is ${name} and I am ${age} years old.`);
```

Number methods

```
None
let value = "100";

console.log(Number(value));
console.log(parseInt("25.5"));
console.log(parseFloat("25.5"));
console.log(isNaN("Hello"));
console.log((10 / 3).toFixed(2));
```

---

11. Scope

Scope determines where a variable can be accessed.

Global scope

```
None
let name = "Divya";

function display() {
    console.log(name);
}
```

Function scope

```
None
function test() {
    var message = "Hello";
    console.log(message);
}

// message is not accessible here
```

Block scope

```
None
if (true) {
    let age = 25;
    const city = "Kerala";
}

// age and city are not accessible here
```

Remember:

- `var` is function-scoped.
- `let` and `const` are block-scoped.

---

12. DOM Manipulation

DOM means Document Object Model.

It represents the HTML document as a tree of objects that JavaScript can access and modify.

Select HTML elements

```
None
document.getElementById("heading");

document.getElementsByClassName("box");

document.getElementsByTagName("p");

document.querySelector(".box");

document.querySelectorAll("p");
```

Change content

```
None
document.getElementById("heading").textContent = "Welcome!";
```

Change HTML

```
None
document.getElementById("content").innerHTML =
    "<strong>Hello!</strong>";
```

Change CSS

```
None
let heading = document.getElementById("heading");

heading.style.color = "blue";
heading.style.fontSize = "30px";
```

Add and remove classes

```
None
let box = document.querySelector(".box");

box.classList.add("active");
box.classList.remove("active");
box.classList.toggle("active");
```

Important: Use `textContent` for plain text. Use `innerHTML` only when HTML markup is intended and the content is trusted or safely handled.

13. JavaScript Events

Events occur when users interact with a webpage.

Common events:

Event	Meaning
click	User clicks an element
dblclick	User double-clicks
mouseover	Pointer moves over an element
mouseout	Pointer leaves an element
keydown	Keyboard key is pressed
input	Input value changes
change	Input selection/value is committed
submit	Form is submitted
load	Resource or page finishes loading

addEventListener()

```
None
<button id="btn">Click Me</button>

<script>
    document.getElementById("btn")
        .addEventListener("click", function() {
            alert("Button clicked!");
        });
</script>
```

Event example: Change heading

```
None
<h2 id="title">Original Heading</h2>
<button id="changeBtn">Change Text</button>

<script>
    document.getElementById("changeBtn")
```

```
        .addEventListener("click", function() {
            document.getElementById("title").textContent =
                "Heading Changed!";
        });
</script>
```

---

14. Form Validation

JavaScript can validate form values before submission.

```
None
<form id="registrationForm">
    <input type="text" id="name" placeholder="Enter name">
    <input type="email" id="email" placeholder="Enter email">

    <button type="submit">Register</button>

    <p id="message"></p>
</form>

<script>
    document.getElementById("registrationForm")
        .addEventListener("submit", function(event) {

            event.preventDefault();

            const name =
document.getElementById("name").value.trim();
            const email =
document.getElementById("email").value.trim();

            if (name === "" || email === "") {
                document.getElementById("message").textContent =
                    "Please fill all fields.";
            } else {
                document.getElementById("message").textContent =
                    "Registration successful!";
            }
        });
</script>
```

`event.preventDefault()` prevents the form's default submission behaviour.

---

15. JSON

JSON stands for JavaScript Object Notation.

It is a text format used to exchange data between applications, especially APIs.

Example JSON:

```
None
{
    "name": "Divya",
    "age": 25,
    "course": "JavaScript"
}
```

JSON.parse()

Converts JSON text into a JavaScript value.

```
None
const jsonText = '{"name":"Divya","age":25}';

const student = JSON.parse(jsonText);

console.log(student.name);
```

JSON.stringify()

Converts a JavaScript value into JSON text.

```
None
const student = {
    name: "Divya",
    age: 25
};

const jsonText = JSON.stringify(student);

console.log(jsonText);
```

Connection to your previous Postman work

You already used this:

```
None
const response = pm.response.json();

if (response.accessToken) {
    pm.environment.set("accessToken", response.accessToken);
}
```

This is a practical example of reading API response data, checking a property and storing a value.

---

16. Error Handling

JavaScript uses `try`, `catch` and `finally` to handle errors.

```
None
try {
    let result = JSON.parse("invalid json");
    console.log(result);
}
catch (error) {
    console.log("Something went wrong:", error.message);
}
finally {
    console.log("Execution completed.");
}
```

- `try`: Code that may produce an error.
- `catch`: Handles the error.
- `finally`: Runs whether an error occurs or not.

---

17. Modern JavaScript (ES6+)

Destructuring

```
None
const student = {
    name: "Divya",
```

```
    age: 25
};

const { name, age } = student;

console.log(name, age);
```

Array destructuring:

```
None
const colors = ["Red", "Blue"];

const [first, second] = colors;
```

Spread operator

Expands values from an iterable or object.

```
None
const first = [1, 2];
const second = [...first, 3, 4];

console.log(second);
```

Rest parameters

Collects remaining arguments into an array.

```
None
function add(...numbers) {
    return numbers.reduce((sum, n) => sum + n, 0);
}

console.log(add(10, 20, 30));
```

Default parameters

```
None
function greet(name = "Guest") {
    console.log(`Hello ${name}`);
}
```

```
greet();
greet("Divya");
```

---

18. Asynchronous JavaScript

Asynchronous operations allow certain tasks to complete without blocking the rest of the program.

setTimeout()

Executes a function after a specified delay.

```
None
setTimeout(function() {
    console.log("Hello after 2 seconds");
}, 2000);
```

Promise

A Promise represents the eventual completion or failure of an asynchronous operation.

```
None
const myPromise = new Promise((resolve, reject) => {
    let success = true;

    if (success) {
        resolve("Operation successful");
    } else {
        reject("Operation failed");
    }
});

myPromise
    .then(result => console.log(result))
    .catch(error => console.log(error));
```

async and await

```
None
async function getData() {
```

```
    try {
        const response = await
fetch("https://jsonplaceholder.typicode.com/users");

        if (!response.ok) {
            throw new Error("Request failed");
        }

        const data = await response.json();

        console.log(data);
    }
    catch (error) {
        console.log(error.message);
    }
}

getData();
```

- `async` declares an asynchronous function.
- `await` waits for a Promise to settle inside an async function.
- `fetch()` makes an HTTP request.
- `response.json()` reads and parses the response body.

---

19. Quick Revision Sheet

Concept	Remember
Variable	Stores data
`let`	Reassignable, block-scoped
`const`	Not reassignable, block-scoped
`typeof`	Checks a value's type
`===`	Strict equality
`&&`	Logical AND
`if...else`	Decision making

`for`	Repeated execution
Function	Reusable code
Array	Ordered collection
Object	Key-value collection
`map()`	Transforms array values
`filter()`	Selects matching values
DOM	Represents HTML document
Event	User/browser interaction
`addEventListener()`	Registers an event handler
`JSON.parse()`	JSON text to JavaScript value
`JSON.stringify()`	JavaScript value to JSON text
Promise	Represents an asynchronous result
`async/await`	Works with Promises

---

20. Important Interview Questions

1. What is JavaScript?
2. What is the difference between `var`, `let` and `const`?
3. What are primitive and non-primitive data types?
4. What is the difference between `==` and `===`?
5. What is the difference between `null` and `undefined`?
6. What is the difference between `if...else` and `switch`?
7. What are the different types of loops?
8. What is a function?
9. What is the difference between parameters and arguments?
10. What is an arrow function?
11. What is the difference between an array and an object?
12. What are `map()`, `filter()` and `reduce()`?
13. What is the DOM?
14. What is an event listener?
15. What is event.preventDefault()?
16. What is JSON?
17. What is the difference between JSON.parse() and JSON.stringify()?
18. What is a Promise?

19. What are async and await?
20. What is error handling?

---

21. Practical Revision Checklist

- Declare variables using `let` and `const`.
- Identify JavaScript data types.
- Perform arithmetic and logical operations.
- Write if/else and switch statements.
- Practice all major loops.
- Create functions with parameters and return values.
- Create and manipulate arrays.
- Practice map, filter and reduce.
- Create objects and access their properties.
- Use string and number methods.
- Understand variable scope.
- Select and modify HTML elements through the DOM.
- Handle button click and input events.
- Validate a registration form.
- Parse and stringify JSON.
- Handle errors with try/catch.
- Understand Promises and async/await.
- Build an interactive webpage.

Assignment 2 completion goal: Independently write JavaScript programs, manipulate webpage elements, validate forms and understand how JavaScript works with API data.